2026

Lab7_Report

MEDICAL IMAGE AND SIGNAL PROCESSING LABORATORY

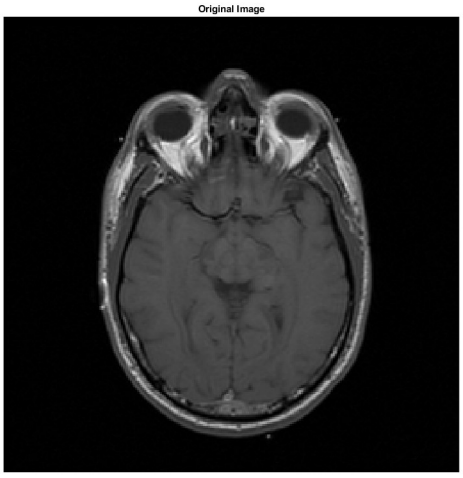
AMITIS MIRABEDINI – NEGIN RAKI

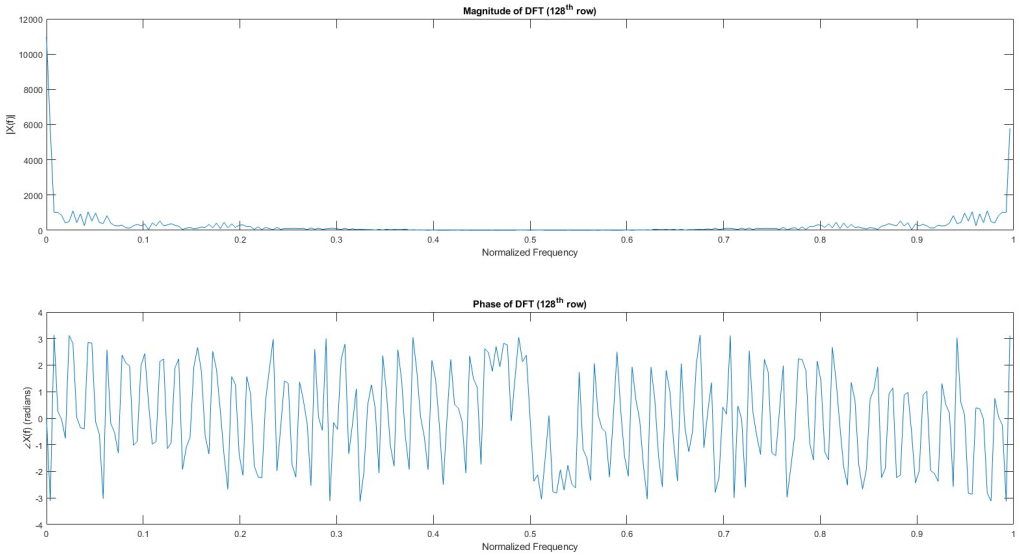
402102562 | 402101758

Question 1

Using the script below, we have obtained the required results

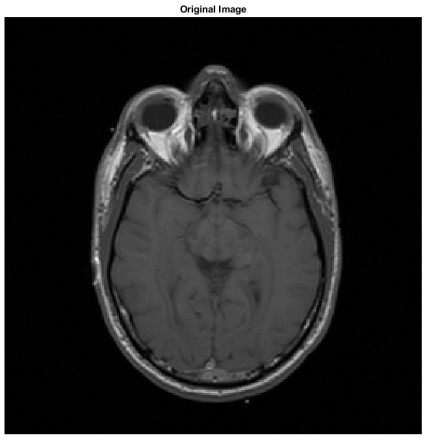
```
%% Q1
IMAGE = imread("E:\6th Semester\MISP Lab\MyLab\LAB7\Lab 7_data\S1_Q1_utils\t1.jpg");
figure;
imshow(IMAGE);
title('Original Image');

first_slice = IMAGE(:,:,1);
rowfft128 = double(first_slice(128, :));
X = fft(rowfft128);

magnitude = abs(X);
phase = angle(X);

N = length(rowfft128);
f = (0:N-1)/N;

figure;

subplot(2,1,1);
plot(f, magnitude);
title('Magnitude of DFT (128^{th} row)');
xlabel('Normalized Frequency');
ylabel('|X(f)|');

subplot(2,1,2);
plot(f, phase);
title('Phase of DFT (128^{th} row)');
xlabel('Normalized Frequency');
ylabel('∠X(f) (radians)');

I = double(first_slice);

logMag = log(1 + abs(fftshift(fft2(I))));

figure;

subplot(1,2,1);
imshow(first_slice);
title('Original Image');

subplot(1,2,2);
imshow(logMag, []);
title('Log-Magnitude of 2D FFT');

```

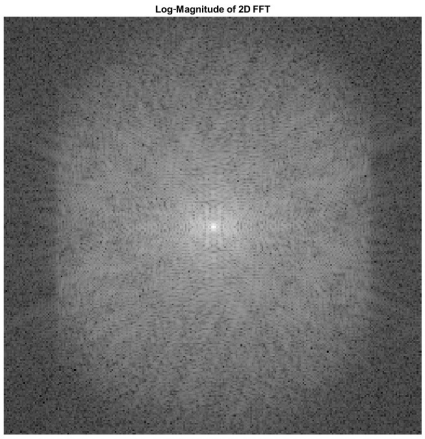




The magnitude spectrum shows that the majority of the signal energy is concentrated at very low frequencies, close to 0, as well as at the highest frequency components near 1. This indicates that the selected row is dominated by low-frequency information, with a smaller contribution from high-frequency components associated with edges or abrupt intensity changes. Such a distribution suggests that the image row consists mainly of smooth variations in intensity, interrupted by a few sharp transitions. The relatively low and uniform magnitude observed in the mid-frequency region implies that there is little texture or fine detail at intermediate spatial scales.

The phase spectrum exhibits rapid oscillations across the entire range of [−π, π]. While the phase information is not as straightforward to interpret visually as the magnitude spectrum, it contains essential spatial information required for accurately reconstructing the original signal. In particular, the phase preserves the positional relationships and structural characteristics of the image. Overall, the DFT analysis suggests that row 128 is primarily composed of low-frequency content, which is characteristic of anatomical structures in MRI images, together with a limited number of sharp intensity changes that likely correspond to edges or boundaries.





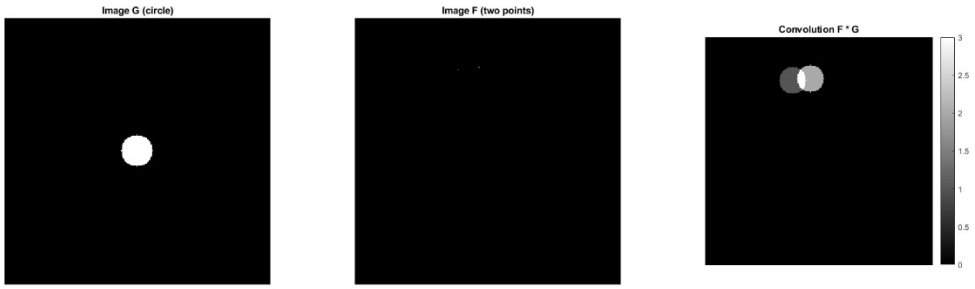
The 2D Fourier spectrum clearly highlights the edges and structural features of the image. It also reveals how the amplitude of the frequency components varies across different spatial directions. Most of the spectral energy is concentrated around the low-frequency region in both the horizontal and vertical directions, indicating that the image is primarily composed of smooth intensity variations rather than fine details. The relatively lower energy at higher frequencies suggests that sharp transitions and edge information occupy only a small portion of the overall frequency content.

Question 2

Using the script below, we obtained the required result:

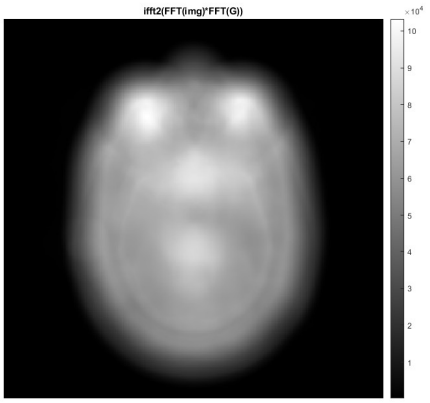
```
%% Q2
N  = 256;
r  = 15;
cx = N/2;
cy = N/2;

[y, x] = ndgrid(1:N, 1:N);
G = (x - cx).^2 + (y - cy).^2 <= r^2;

F = zeros(N, N);

F(50, 100) = 1;
F(48, 120) = 2;

convGF = fftshift(real(ifft2(fft2(double(F)) .* fft2(double(G)))));


subplot(1,3,1);
imshow(G);
title('Image G (circle)');

subplot(1,3,2);
imagesc(F);
axis image off;
colormap gray;
title('Image F (two points)');

subplot(1,3,3);
imagesc(convGF);
axis image off;
colormap gray;
colorbar;
title('Convolution F * G');


img2 = imread("E:\6th Semester\MISP Lab\MyLab\LAB7\Lab 7_data\S1_Q2_utils\pd.jpg");

img2_slice1 = double(img2(:, :, 1));

FG = fft2(double(G));
Fslice1 = fft2(img2_slice1);
conv_freq_real =fftshift(real(ifft2(Fslice1 .* FG)));

figure;

subplot(1,2,1);
```

```
figure;

subplot(1,2,1);
imagesc(img2_slice1);
axis image off;
colormap gray;
title('Original Image');

subplot(1,2,2);
imagesc(conv_freq_real);
axis image off;
colormap gray;
title('ifft2(FFT(img)*FFT(G))');
colorbar;
```



The resulting figure is consistent with our expectations. Since the two images were convolved, we expect the shape of the first white circle to appear within the second black square. The difference in intensity levels is due to the initial intensity values assigned to the two dots in the original images.

Next, an MRI slice was selected from the dataset and convolved with the same simple white circle. This operation was performed to examine the effect of the convolution on a real medical image. The resulting image is shown in the following figure.

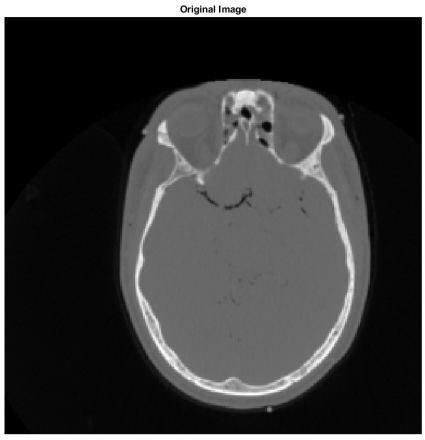


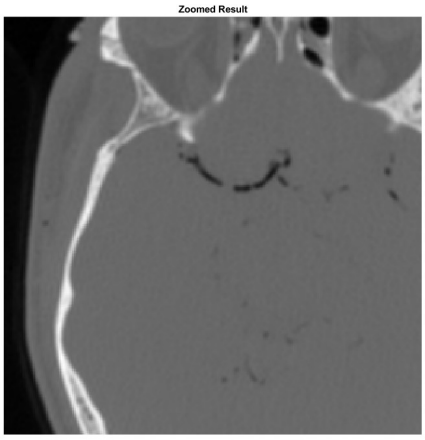
The output demonstrates that convolving the MRI slice with the white circular mask results in a blurred version of the original image. This behavior is expected because the circular mask functions as a smoothing or low-pass filtering kernel. During convolution, the intensity of each pixel is influenced by the values of its neighboring pixels, effectively averaging local regions of the image. As a result, rapid intensity variations and high-frequency details, such as edges and fine structures, are reduced, leading to the observed blurring effect.

Question 3

Using the script below, we obtained the results:

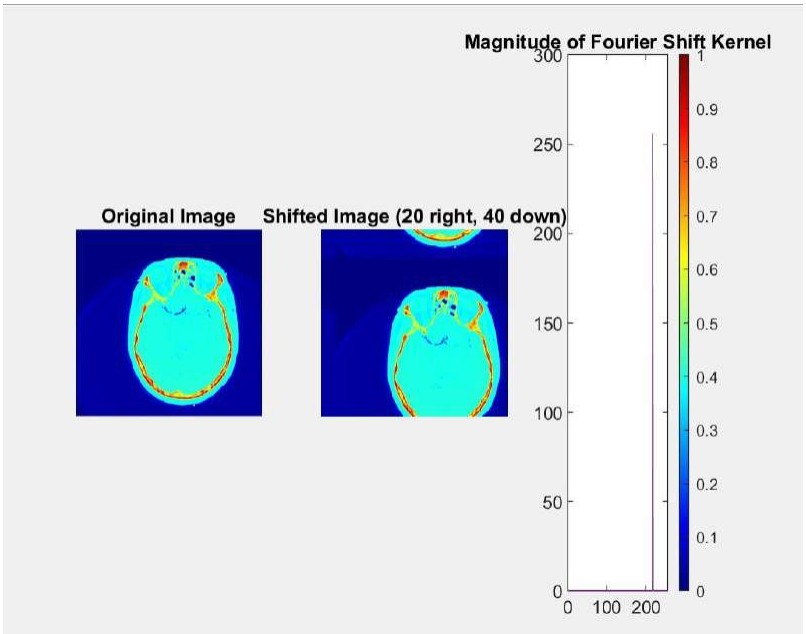
```
%% Q3

img3 = imread("E:\6th Semester\MISP Lab\MyLab\LAB7\Lab 7_data\S1_Q3_utils\ct.jpg");

img3_slice1 = double(img3(:,:,1));

% FFT of image and shift DC to center
FFT_img3 = fftshift(fft2(img3_slice1));

% Zero-padding in frequency domain
FFT_padded = padarray(FFT_img3,[128 128]);

% Back to spatial domain
zoomed_img = ifft2(ifftshift(FFT_padded));

figure;

subplot(1,2,1);
imshow(img3_slice1,[]);
title('Original Image');

subplot(1,2,2);
imshow(abs(zoomed_img(129:384,129:384)),[]);
title('Zoomed Result');
```





Question 4_1

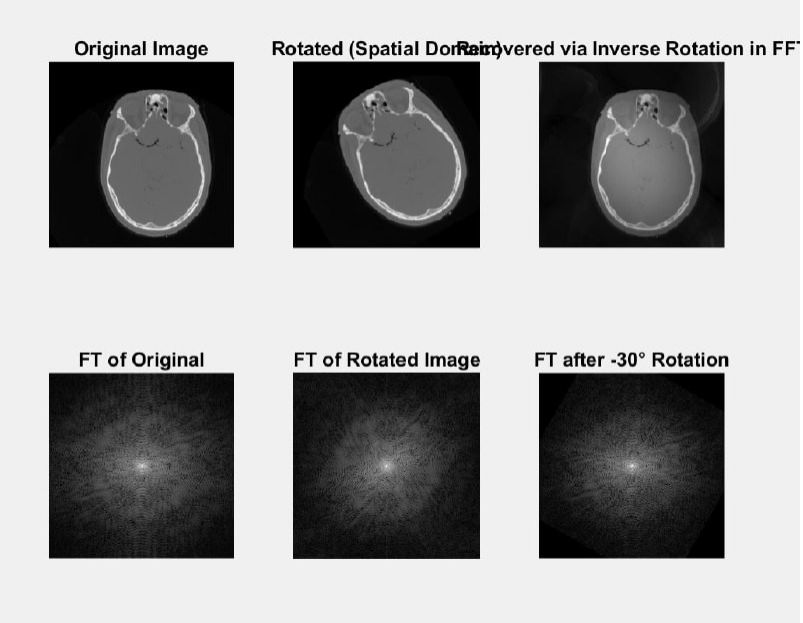
```
%% Loading Data
clc;
IMG = imread("C:\Users\rakyn\OneDrive\Desktop\TERM6\MISP_LAB\LAB7\Data\S1_Q4_utils\ct.jpg");
IMG = im2double(IMG);
IMG = IMG(:,:,1);

%% Shifting Image
[M,N,C]=size(IMG);

delta_x=20;
delta_y=40;

%frequency grids
[u,v]=meshgrid(0:N-1,0:M-1);
u = ifftshift(u - floor(N/2));
v = ifftshift(v - floor(M/2));

%shift kernel
shift_kernel = exp(-1i*2*pi*(u*delta_x/N + v*delta_y/M));

shifted_img=zeros(size(IMG));
for c = 1:C
    F = fft2(IMG(:,:,c));
    F_shifted = F .* shift_kernel;
    shifted_img(:,:,c) = real(ifft2(F_shifted));
end
figure;
subplot(1,3,1);
imshow(IMG);
title('Original Image');

subplot(1,3,2);
imshow(shifted_img);
title('Shifted Image (20 right, 40 down)');

subplot(1,3,3);
plot(abs(fft(shift_kernel)));
title('Magnitude of Fourier Shift Kernel');
colormap jet; colorbar;
```



Translation in the frequency domain is achieved using the Fourier Shift, that says that a spatial shift corresponds to a linear phase shift in the frequency domain. The implementation creates a 2D frequency grid (meshgrid) and defines the phase shift kernel according to the formula above. This kernel is multiplied element-wise with the image's Fast Fourier Transform (FFT). Finally, the Inverse FFT ((ifft2)) is applied to recover the translated spatial image.

Question 4_2

```
%% P4_2_Rotating IMG

thta=30;
img_rotated = imrotate(IMG, thta, 'bilinear', 'crop');

% fft of rotated image
f_rIMG=fft2(ifftshift(img_rotated));
f_rIMG_shifted=fftshift(f_rIMG);

% undo rotation in freq domain
F_unrotated = imrotate(f_rIMG_shifted, -thta, 'bilinear', 'crop');

F_unrotated_unshifted = ifftshift(F_unrotated);
img_recovered = fftshift(real(ifft2(F_unrotated_unshifted)));

% Step 6: Visualization
figure;

subplot(2,3,1);
imshow(IMG); title('Original Image');

subplot(2,3,2);
imshow(img_rotated); title('Rotated (Spatial Domain)');

subplot(2,3,3);
imshow(img_recovered, []); title('Recovered via Inverse Rotation in FFT');

subplot(2,3,4);
imagesc(log(1 + abs(fftshift(fft2(IMG))))); axis image off;
title('FT of Original');

subplot(2,3,5);
imagesc(log(1 + abs(f_rIMG_shifted))); axis image off;
title('FT of Rotated Image');

subplot(2,3,6);
imagesc(log(1 + abs(F_unrotated ))); axis image off;
title('FT after -30° Rotation');
colormap gray;
```



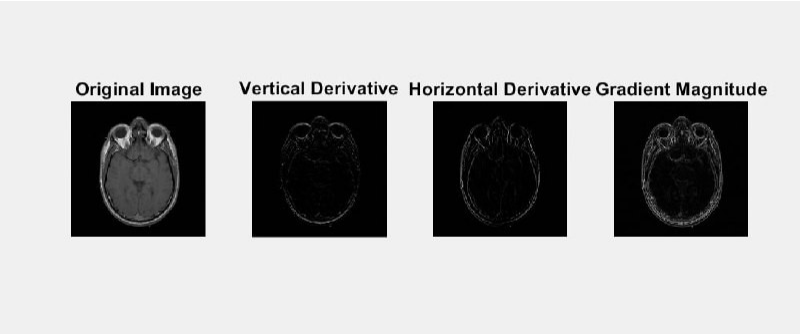
The Fourier transform of a rotated image is the Fourier transform of the original image rotated by the same angle.

First,the spatial image is rotated by a specific angle (like 30 degrees) using imrotate.Then,the FFT of the rotated image is calculated.

The spectrum is then rotated back (-30 degrees).Applying the Inverse FFT to this inverse-rotated spectrum successfully recovers the original unrotated image. This shows us a rotational property of the Fourier Transform.

Question 5

```
%% Loading image p5
clc;
IMG = imread("C:\Users\rakyn\OneDrive\Desktop\TERM6\MISP_LAB\LAB7\Data\S1_Q5_utils\t1.jpg");
IMG = im2double(IMG);
IMG = IMG(:,:,1);

clc;

figure;
subplot(1, 4, 1);
imshow(IMG);
title('Original Image');

% Vertical derivative (dy)
Ver_img = circshift(IMG, [-1, 0]) - circshift(IMG, [1, 0]);

% Horizontal derivative (dx)
Hor_img = circshift(IMG, [0, -1]) - circshift(IMG, [0, 1]);

% Step 3: Compute Gradient Magnitude
Gradient = sqrt(double(Hor_img).^2 + double(Ver_img).^2);


% Display the Ver_img derivative
subplot(1, 4, 2);
imshow(Ver_img);
title('Vertical Derivative');

% Display the horizontal derivative
subplot(1, 4, 3);
imshow(Hor_img);
title('Horizontal Derivative');

% Display the gradient magnitude
subplot(1, 4, 4);
imshow(Gradient);
title('Gradient Magnitude');
```



Edge detection relies on identifying areas of rapid intensity change. This part manually computes the image gradient using central finite differences via the circshift function.

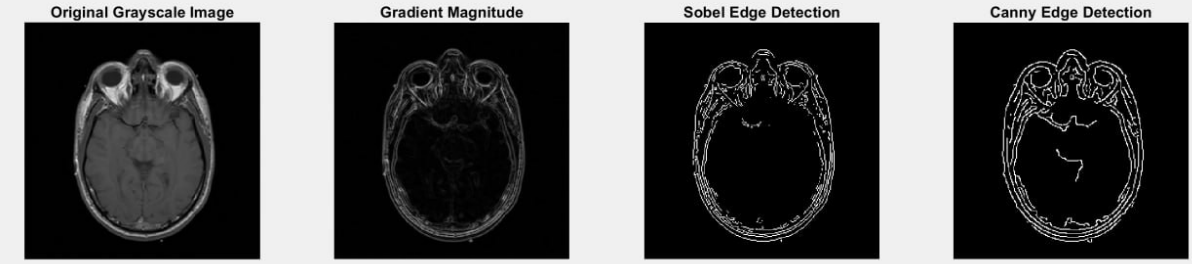
Vertical Gradient ((G_y)): Computes the difference between the pixel above and below.

Horizontal Gradient ((G_x)): Computes the difference between the pixel left and right.

Gradient Magnitude: Calculates the overall edge strength using the Euclidean distance.

Question 6

```
%% P6

clc;
% Apply Sobel edge detection
edges_sobel = edge(IMG, 'sobel');

% Apply Canny edge detection
edges_canny = edge(IMG, 'canny', [0.05 0.2]);

% Plot results
figure;

subplot(1, 4, 1);
imshow(IMG);
title('Original Grayscale Image');

subplot(1, 4, 2);
imshow(Gradient);
title('Gradient Magnitude');

subplot(1, 4, 3);
imshow(edges_sobel);
title('Sobel Edge Detection');

subplot(1, 4, 4);
imshow(edges_canny);
title('Canny Edge Detection');
```



1. Sobel Edge Detection
   The Sobel method computes a 2D spatial gradient by convolving the image with
   two (3 * 3) kernels. These kernels give more weight to the central pixels, providing a smoothing effect that reduces high-frequency noise compared to simple finite differences.
2. Canny Edge Detection
   Canny is a multi-stage algorithm designed to find optimal, continuous, and thin
   edges with heavily denoising. It is significantly more robust than Sobel.
   Algorithm Steps:
   Gaussian Smoothing: Convolves the image with a Gaussian filter to remove noise.

Non-Maximum Suppression: Scans the image and checks if a pixel is a local maximum in the gradient direction . If not, it is set to 0, ensuring thin, single-pixel edges.